

day05 课堂笔记

0. 复习

希望老师讲完每个知识点可以安排一点点时间去练习一下

这一章节暂时没有太大问题.

- 1 标识符规则：由字母, 数字和下划线组成, 不能以数字开头
- 2 变量名不能和关键字重名.
- 3 关键字是系统定义好的标识符, 具有特殊的作用.
- 4 数据类型: `int`(整型) `float`(浮点型) `str`(字符串类型)
`bool`(布尔类型, `True False`) `list`(列表) `tuple`(元组) `dict`(字典)
- 5 `type()` 函数可以查看变量的数据类型
- 6 输入: `input()` `--> str` 类型转换
- 7 输出: `print()` `%s`--字符串 `%d` -- `int` `%f`--
`float` `%--%%`
- 8 `f-string` {变量}
- 9
- 10 逻辑运算符
- 11 `and` 逻辑与 一假为假
- 12 `or` 逻辑或 一真为真
- 13 `not` 逻辑非 取反

```
14 比较运算符
15      ==      !=
16
17 判断 if elif else
18 循环 while
19      for xx in 容器 / range():
20          pass
21      range(a, b, step)  [a, b) 之间的整数
22      break 终止循环
23      continue 跳过本次循环, 继续下次循环
24      循环 else 结构: 循环不是被 break 终止的时候会
    执行 else 中的代码
25
26 str: 下标 切片
27 list: insert append extend ...

28 tuple: 元组中的数据不能修改 查询的方法 index()
29 dict: key:value 键值对的形式
30
31 函数:
32 定义: 使用 def 关键字,
33 def 函数名(形参列表):
34     函数代码
35     return 数据
36     pass
37
38 函数调用, 才会执行函数中的代码
```

- 39 函数名(实参列表)
- 40 局部变量：在函数内部定义的变量,只能在当前函数内部使用, 如果想要将这个变量在函数外部使用, ①可以 `return` 将这个变量返回 ② 使用 `global`, 将其变为全局变量
- 41 全局变量：在任意地方可以访问, 想要在函数内部修改全局变量的值, 使用 `global` 声明

1.函数传参的两种形式[掌握]

```
1 def func(a, b, c):
2     print(f"a: {a}")
3     print(f"b: {b}")
4     print(f"c: {c}")
5
6
7 # 位置传参, 按照形参的位置顺序将实参的值传递给形参
8 func(1, 2, 3)
9 # func(3, 1, 2)
10 # 关键字传参, 指定实参给到哪个形参, 注意点: 关键字必须是函数的形参名
11 func(a=10, b=20, c=30)
12 # func(c=10, a=20, b=30)
13 # 混合使用, 先写位置传参 再写关键字传参
14 # func(10, b=20, c=30)
15 # func(a=10, 20, 30) # 代码会报错
16 # func(10, a=30, b=20) # 代码会报错
17
```

```
15 func(a=10, 20, 30)
16
17
18
```

错误的写法

关键字实参要写在位置实参的后边

```
Run: 02-函数传参的两种形式 x
C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day05/02-函数传参的两种形式.py
File "C:/Users/n1/Desktop/43/day05/02-函数传参的两种形式.py", line 15:
    func(a=10, 20, 30)
                ^
SyntaxError: positional argument follows keyword argument

Process finished with exit code 1
```

2. 函数形参

2.1 缺省参数(默认参数)

```
1 # print()
2 # 缺省参数, 形参. 在函数定义的时候, 给形参一个默认值, 这个形参就是缺省参数,
3 # 注意点: 缺省参数要写在普通参数的后边
4 # 特点: 在函数调用的时候, 如果给缺省参数传递实参值, 使用的是传递的实参值, 如果没有传递, 使用默认值
5
6
7 def func(a, b, c=10): # 形参c 称为缺省形参
8     print(f'a: {a}')
9     print(f'b: {b}')
10    print(f'c: {c}')
11
12
13 func(1, 2) # 没有给c 传递实参, 使用默认值10
14 func(1, 2, 3) # 给c 传递实参值, 使用传递的数据3
15
16
17 # def fun1(a=1, b, c): # 代码报错, 语法错误
18 #     pass
```

```
a: 1
b: 2
c: 10
a: 1
b: 2
c: 3

Process finished with exit code 0
```

2.2 不定长参数

- 1 注意点: 函数定义中的 args 和 kwargs 可以是任意的形参变量, 不过习惯使用 args 和 kwargs.

```
8 def func(args, kwargs): # 两个普通的形参
9     print(args)
10    print(kwargs)
11
12
13    func(1, 2)
14    func(args=2, kwargs=1)
15
16    print('*' * 30)
17    print('*' * 30)
18
19
20    # 在形参前边加上一个*, 该形参变为不定长元组形参, 可以接收所有的位置实参, 类型是元组
21    # 在形参前边加上两个**, 该形参变为不定长字典形参, 可以接收所有的关键字实参, 类型是字典
22    def func(*args, **kwargs):
23        print(args)
24        print(kwargs)
25
26
27    func(1, 2, 3, 3, 4, 5)
28    func(a=1, b=2, c=3, d=4)
29    func(1, 2, 3, a=4, b=5, d=6)
```

```
1
2
2
1
*****
*****
(1, 2, 3, 3, 4, 5)
{}
()
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
(1, 2, 3)
{'a': 4, 'b': 5, 'd': 6}

Process finished with exit code 0
```

2.3 函数形参的完整格式

```
1 # 普通形参 缺省形参 不定长元组形参 不定长字典形参
2 def func(a, b=1): # 先普通再 缺省
3     pass
4
5
6 def func1(a, b=1, *args): # 语法上不会报错, 但是缺省参数不能使用默认值
7     print('a', a)
8     print('b', b)
9     print(args)
10
11
12 def func2(a, *args, b=1): # 普通形参 不定长元组形参 缺省形参
13     print('a', a)
14     print('b', b)
15     print(args)
16
17
18 def func3(a, *args, b=1, **kwargs): # 普通形参 不定长元组形参 缺省形参 不定长字典形参
19     pass
20
21
22 # func1(1, 2, 3, 4)
23 # func2(1, 2, 3, 4)
24 func2(1, 2, 3, 4, b=10)
```

3. 拆包[掌握]

- 注意点: 容器中元素数据的个数需要和变量的个数保持一致.

```
1  # 组包, 将多个数据值, 组成元组, 给到一个变量.
2  a = 1, 2, 3
3  print(a) # (1, 2, 3)
4
5
6  def func():
7      return 1, 2 # 组包
8
9
10 # 拆包: 将容器的数据分别给到多个变量, 需要注意: 数据的个数和变量的个数要保持一致
11 b, c, d = a # 拆包
12 # print(b, c, d)
13
14 e, f = func()
15 print(e, f)
16
17 my_list = [10, 20]
18 a, b = my_list
19 print(a, b)
20 my_dict = {'name': 'isaac', 'age': 18}
21 a, b = my_dict # key值
22 print(a, b)
```

```
16 # 方法三, python中的使用 组包和拆包
17 a, b = b, a          1. 组包 (20, 10)
18 print(a, b)          2. 拆包 a, b = (20, 10)
19
```

Run: 09-拆包应用-交换两个变量的值 ×

C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day05/09-拆包应
20 10

Process finished with exit code 0

4. 引用[理解]

```

1 # 可以使用 id() 查看变量的引用, 可以将id 值认为是内存地址的别名
2 # python中数据值的传递的是引用
3 # 赋值运算符可以改变变量的引用
4
5 # 将数据10 存储到变量a 中, 本质是将数据10 所在内存的引用地址保存到变量a 中
6 a = 10
7 # 将变量a中保存的引用地址给到b
8 b = a
9 print(a, b) # 使用print函数打印变量a 和b 引用中存储的值
10 print(id(a), id(b))
11 a = 20
12 print(a, b)
13 print(id(a), id(b))
14

```

Run: 10-引用 x

```

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day05/10-引用.py
10 10
140733938063296 140733938063296
20 10
140733938063616 140733938063296

```

```

5 # 将数据10 存储到变量a 中, 本质是将数据10 所在内存的引用地址保存到变量a 中
6 a = 10
7 # 将变量a中保存的引用地址给到b
8 b = a
9 print(a, b) # 使用print函数打印变量a 和b 引用中存储的值
10 print(id(a), id(b))
11 a = 20
12

```



Run: 10-引用 x

```

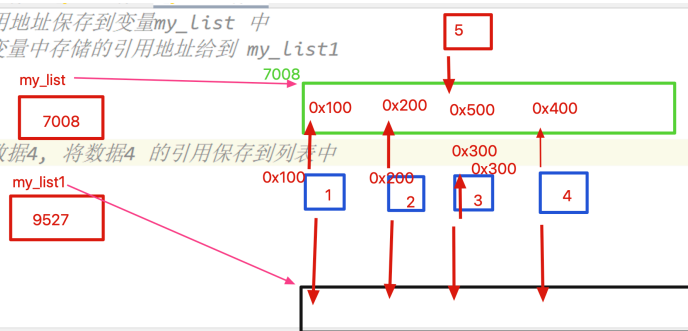
C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day05/10-引用.py
10 10
140733938063296 140733938063296

```

```

1 my_list = [1, 2, 3] # 将列表的引用地址保存到变量my_list 中
2 my_list1 = my_list # 将my_list 变量中存储的引用地址给到 my_list1
3 print(my_list, id(my_list))
4 print(my_list1, id(my_list1))
5
6 my_list.append(4) # 向列表中添加数据4, 将数据4 的引用保存到列表中
7 print(my_list, id(my_list))
8 print(my_list1, id(my_list1))
9
10 my_list[2] = 5
11 print(my_list, id(my_list))
12 print(my_list1, id(my_list1))
13 my_list1 = [1, 2, 3, 4]

```



Run: 11-引用2 x

```

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day05/11-引用2.py
[1, 2, 3] 2590251667008
[1, 2, 3] 2590251667008
[1, 2, 3, 4] 2590251667008
[1, 2, 3, 4] 2590251667008
[1, 2, 5, 4] 2590251667008
[1, 2, 5, 4] 2590251667008

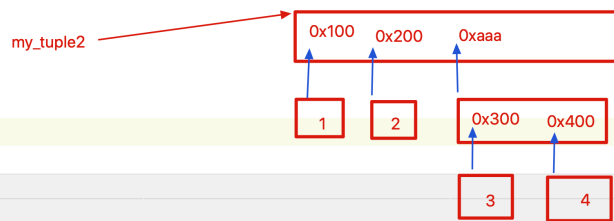
Process finished with exit code 0

```

5. 可变与不可变类型

```
1 # 类型的可变与不可变: 在不改变变量引用的前提下, 能否改变变量中引用中的数据,
2 # 如果能改变是可变类型, 如果不能改变, 是不可变类型
3 # int float bool str list tuple dict
4 # 不可变类型: int float bool str tuple
5 # 可变类型: list dict
6
```

```
31 print('*' * 30)
32 my_tuple2 = (1, 2, [3, 4])
33 print(my_tuple2)
34 my_tuple2[2][1] = 10
35 print(my_tuple2)
36
```



Run: 12-类型的可变与不可变

```
2060011163136 2060011192000
*****
(1, 2, [3, 4])
(1, 2, [3, 10])
```

6. 引用做函数参数注意点


```
1  # 函数传参传递的也是引用
2  my_list = [1, 2, 3] # 全局变量
3
4
5  def func1(a):
6      a.append(4)
7
8
9  def func2():
10     # 为啥不加global, 因为没有修改 my_list 中存的引用值
11     my_list.append(5)
12
13
14  def func3():
15     global my_list
16     my_list = [1, 2, 3] # 修改全局变量的值
17
18
19  def func4(a):
20     # += 对于列表来说, 类似列表的extend方法, 不会改变变量的引用地址
21     a += a # a = a + a, 修改了a变量a的引用
22     # print(a)
23
24
25  func1(my_list) # [1, 2, 3, 4]
26  func2() # [1, 2, 3, 4, 5]
27  func3() # [1, 2, 3]
28  print(my_list)
29
30  b = 10 # 不可变类型
31  func4(b)
32  print(b) #
33
34  func4(my_list)
35  print(my_list) # [1, 2, 3, 1, 2, 3]
```

7. 函数应用: 学生管理系统

```
1  增删改查操作
2  存储学生信息：
3      姓名
4      年龄
5      班级
6      性别
7      ...
8      {'name': 'isaac', 'age': 18, ...}
9
10 存储所有的学生信息：
11      列表：[{}, {}, {}, ....]
12
13 功能菜单, 业务框架.
```

1. 添加学生
2. 删除学生
3. 修改学生信息
4. 查询单个学生信息
5. 查询所有学生信息
6. 退出系统

请选择要进行的操作编号：

搭建业务框架

```
1  def show_menu():
2      print('1. 添加学生')
3      print('2. 删除学生')
4      print('3. 修改学生信息')
5      print('4. 查询单个学生信息')
6      print('5. 查询所有的学生信息')
7      print('6. 退出系统')
8
9
10 while True:
11     show_menu()
12     opt = input('请输入用来选择的操作编号:')
13     if opt == '1':
14         print('1. 添加学生')
15     elif opt == '2':
16         print('2. 删除学生')
17     elif opt == '3':
18         print('3. 修改学生信息')
19     elif opt == '4':
20         print('4. 查询单个学生信息')
21     elif opt == '5':
22         print('5. 查询所有的学生信息')
23     elif opt == '6':
24         print('欢迎下次使用本系统.....')
25         break
```

```
26         else:
27             print('输入有误,请再次输入')
28             continue
29
30     input('..... 回车键继续操作.....')
31
```

添加学生信息

1. 通过 `input` 函数获取学生的信息, 姓名, 年龄, 性别
 2. 将学生信息转换为字典进行保存
 3. 将这个学生字典添加的列表中
- 代码优化:
- 假设学生的名字都不能重复, 在添加学生的时候, 进行判断, 如果学生名字已经存在, 则不添加.

删除 / 修改 / 查询学生信息

- 1 都使用 name 对学生进行判断
- 2
- 3 1. 使用 input 获取要删除 /修改/查询 的学生姓名
- 4 2. 判断学生信息是否存在
- 5 3. 学生存在,对学生进行 删除 /修改/查询 操作
- 6 4. 学生信息不存在,直接结束